

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение
Высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий и искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о выполнении курсовой работы
по дисциплине «Дискретная математика»
«Создание калькулятора «большой» конечной арифметики»
Вариант 26

Выполнил студент группы №5130201/40002

Казаров А.А.

Работу принял

Востров А.В.

«____» _____ 2026 г.

Санкт-Петербург
2026

Содержание

Введение	4
1 Математическое описание	5
1.1 Сложение в «малой» конечной арифметике	5
1.2 Умножение в «малой» конечной арифметике	6
1.3 Переход к «большой» конечной арифметике. Реализация переноса	7
1.4 Отрицательные числа и действия вычитания и деления на множестве $\mathbb{Z}_8^8$	9
2 Особенности реализации	12
2.1 Класс Rules	12
2.1.1 Конструктор Rules	13
2.1.2 Метод eval_plus	18
2.1.3 Метод eval_plus_c	19
2.1.4 Метод eval_times	19
2.1.5 Метод eval_times_c	19
2.2 Класс Finit	20
2.2.1 Конструктор класса Finit	22
2.2.2 Перегрузка оператора +	22
2.2.3 Метод eval_dop	24
2.2.4 Перегрузка операторов <, ==, <=	24
2.2.5 Перегрузка оператора −	26
2.2.6 Сумма двух чисел	27
2.2.7 Разность двух чисел	28
2.2.8 Произведение двух чисел	28
2.2.9 Деление двух чисел	29
2.2.10 Вывод числа на консоль	32
3 Результаты работы программы	33
Заключение	47

Введение

Цель курсовой работы – сформировать калькулятор «большой» конечной арифметики $\langle \mathbb{Z}_8^8; +, * \rangle$ (8 разрядов) для четырех действий $(+, -, *, \div)$ на основе «малой» конечной арифметики $\langle \mathbb{Z}_8; +, * \rangle$. «Малая» конечная арифметика $\langle \mathbb{Z}_8; +, * \rangle$ задана с помощью «правила +1» для варианта 26 (табл. 1). В этой арифметике заданы две бинарные операции «+», «*», для обеих операций выполняются свойства коммутативности, ассоциативности, выполняется свойство дистрибутивности «*» относительно «+», задана аддитивная единица «a» и мультипликативная единица «b». Также выполняется свойство: $\forall x \quad x * a = a$. «Большая» конечная арифметика на основе заданной «малой» – это множество всевозможных элементов из $\mathbb{Z}_8 \times \mathbb{Z}_8 \times \dots$, степень зависит от разрядности чисел в арифметике. В данном случае арифметика восьмиразрядная, поэтому «большая» конечная арифметика – это множество $\mathbb{Z}_8^8$ с заданными на нем операциями сложения «+» и умножения «*», обладающими теми же свойствами, что и в «малой» арифметике. Итак, необходимо производить вычисления над заданным множеством.

Таблица 1: правило +1

a	b	c	d	e	f	g	h
b	d	h	c	f	a	e	g

Задачи курсовой работы:

1. Определить сложение чисел в «малой» конечной арифметике.
2. Определить умножение чисел в «малой» конечной арифметике.
3. Реализовать программно сложение на множестве $\mathbb{Z}_8^8$.
4. Реализовать программно вычитание на множестве $\mathbb{Z}_8^8$.
5. Реализовать программно умножение на множестве $\mathbb{Z}_8^8$.
6. Реализовать программно деление на множестве $\mathbb{Z}_8^8$.
7. Реализовать пользовательское меню для вычисления над множеством $\mathbb{Z}_8^8$.
8. Реализовать защиту от некорректного пользовательского ввода.

1 Математическое описание

Итак, задана «малая» конечная арифметика с приведенным выше «правилом $+1$ ». Это правило устанавливает линейный порядок на заданном множестве $\mathbb{Z}_8 = \{a, b, c, d, e, f, g, h\}$.

Можно представить данное множество с помощью диаграммы Хассе, она имеет форму прямолинейной цепи в силу линейности порядка (рис. 1)



Рис. 1: диаграмма Хассе

Во всяком конечном непустом частично упорядоченном множестве существует минимальный элемент. В данном случае a – минимальный элемент.

1.1 Сложение в «малой» конечной арифметике

Для того, чтобы определить сложение в «малой» арифметике, необходимо знать результат сложения любых двух его элементов. Существует аддитивная единица a , следовательно, результат ее сложения с любым другим элементом уже известен. Результат сложения элемента b с любым другим элементом также известен из исходного условия. Для того, чтобы заполнить остальные строки таблицы сложения (табл. 2), необходимо воспользоваться уже полученными данными. Например, $c + a = c$ из условия аддитивности элемента a , $c + b = h$

из данного условия в варианте.

$$c + d = c + (b + b) = (c + b) + b = h + b = g;$$

$$c + c = b + d + b + d = b + b + b + b + d = c + b + b + b = h + b + b = g + b = e;$$

здесь используется свойство ассоциативности для «+» и результат ранее найденной операции. Таким образом, можно заполнить все остальные ячейки таблицы. Для упрощения легко заметить, что из коммутативности сложения «+» следует, что таблица обладает симметрией относительно главной диагонали.

Таблица 2: таблица сложения $+_s$

$+_s$	a	b	c	d	e	f	g	h
a	a	b	c	d	e	f	g	h
b	b	d	h	c	f	a	e	g
c	c	h	e	g	b	d	a	f
d	d	c	g	h	a	b	f	e
e	e	f	b	a	h	g	c	d
f	f	a	d	b	g	e	h	c
g	g	e	a	f	c	h	d	b
h	h	g	f	e	d	c	b	a

1.2 Умножение в «малой» конечной арифметике

Заданное свойство аддитивной единицы: $\forall x \quad x * a = a$ позволяет заполнить первую строку в таблице умножения (табл. 3). Свойство мультипликативной единицы b позволяет заполнить вторую строку. Для заполнения третьей строки необходимо воспользоваться свойством дистрибутивности операции «*» по отношению к операции «+». Первые две ячейки заполняются аналогично предыдущим ячейкам: $c * a = a$, $c * b = c$; далее по дистрибутивности:

$$c * c = c * (d + b) = c * (b + b + b) = c * b + c * b + c * b = c + c + c = b.$$

$$c * d = c * (b + b) = c * b + c * b = c + c = e$$

Таким образом заполняются все остальные ячейки таблицы. Свойство коммутативности операции «*» снова позволяет найти данным способом лишь для половины таблицы, включая элементы на главной диагонали, остальные ячейки симметричны заполненным.

Таблица 3: Таблица умножения $\ast_s$

$\ast_s$	a	b	c	d	e	f	g	h
a	a	a	a	a	a	a	a	a
b	a	b	c	d	e	f	g	h
c	a	c	b	e	d	g	f	h
d	a	d	e	h	h	e	d	a
e	a	e	d	h	h	d	e	a
f	a	f	g	e	d	b	c	h
g	a	g	f	d	e	c	b	h
h	a	h	h	a	a	h	h	a

1.3 Переход к «большой» конечной арифметике. Реализация переноса

Для того, чтобы перейти к «большой» конечной арифметике, необходимо задать порядок на множестве $\mathbb{Z}_8^8$. Пусть числа упорядочены так, что, например, после числа $aaaaaaaf$ идет число $aaaaaaba$ (для краткости в дальнейшем незначащие нули будут опускаться). Другими словами, должно выполняться следующее: $f + b = ba$. В общем случае, число $\overline{x_7x_6x_5x_4x_3x_2x_1x_0}$ меньше числа $\overline{y_7y_6y_5y_4y_3y_2y_1y_0}$, если $x_7 < y_7$, или $x_7 = y_7 \ \& \ x_6 < y_6$, или $x_7 = y_7 \ \& \ x_6 = y_6 \ \& \ x_5 < y_5$, и так далее, или если $x_7 = y_7 \ \& \ x_6 = y_6 \ \& \ x_5 = y_5 \ \& \ x_4 = y_4 \ \& \ x_3 = y_3 \ \& \ x_2 = y_2 \ \& \ x_1 = y_1 \ \& \ x_0 < y_0$,

По таблице сложения $+_s$ видно, что действительно, $f +_s b = a$. Разряд b взялся из-за переноса. Тогда имеет смысл составить таблицу переносов для сложения $+_c$ (табл. 4)

Таблица 4: Таблица переноса сложения $+_c$

$+_c$	a	b	c	d	e	f	g	h
a	a	a	a	a	a	a	a	a
b	a	a	a	a	a	a	b	a
c	a	a	a	b	b	b	a	a
d	a	a	a	a	b	b	a	a
e	a	a	b	b	b	b	b	b
f	a	b	b	b	b	b	b	b
g	a	a	b	a	b	b	b	b
h	a	a	a	a	b	b	b	b

Из таблицы видно, что $f +_c b = b$. Тогда при сложении двух чисел необходимо учитывать перенос. Например, в общем случае для трехзначных чисел это

выглядит следующим образом:

$$\overline{x_1 y_1} + \overline{x_2 y_2} = \overline{t_3 x_3 y_3}, \text{ где}$$

x_i, y_i, t_i – цифры из множества $\mathbb{Z}_8$,

$$y_3 = y_1 +_s y_2; \quad x_3 = x_1 +_s x_2 +_s (y_1 +_c y_2); \quad t_3 = b, \text{ если } x_1 +_c x_2 = b, \text{ иначе } t_3 = (x_1 +_s x_2) +_c (y_1 +_c y_2).$$

Аналогично для чисел других разрядностей.

Для умножения на множестве $\mathbb{Z}_8^8$ нужно также учитывать разряды переноса, например,

$g * e \neq e$ в смысле арифметики « $\mathbb{Z}_8^8; +, *$ », проверка достигается сложением:

$$g * e = g + g + g + g + g + g = bd + bd + bd = dh + bd = ce.$$

Можно ввести операцию умножения $*_c$ – ее результатом будет разряд переноса при перемножении цифр из $\mathbb{Z}_8$. Например, $g *_c e = c$. Удобнее считать перенос по диаграмме Хассе: она цикличная, и результат перемножения $*_c$ – количество переходов к началу диаграммы при последовательном прибавлении числа, которое умножается.

Из диаграммы Хассе:

$a_a \rightarrow b \rightarrow d \rightarrow c \rightarrow h \rightarrow g \rightarrow e \rightarrow f_a \rightarrow a_b \rightarrow b \rightarrow d \rightarrow c \rightarrow h \rightarrow g \rightarrow e \rightarrow f_b$, и так далее.

В условиях обозначений при вычислении $h *_c c$ необходимо найти, между какими a_i и f_i заключено значение выражения $h +_s h +_s h$. Понятно, что последовательным счетом выяснится, что искомое значение находится между a_b и f_b . Значит, $h *_c c = b$, и $h *_c c = \overline{h *_c c}, h *_s c = bh$. Таким образом составлена таблица (табл. 5)

Таблица 5: Таблица переноса умножения $*_c$

$*_c$	a	b	c	d	e	f	g	h
a	a	a	a	a	a	a	a	a
b	a	a	a	a	a	a	a	a
c	a	a	b	a	d	d	b	b
d	a	a	a	a	b	b	b	b
e	a	a	d	b	h	g	c	c
f	a	a	d	b	g	e	h	c
g	a	a	b	b	c	h	c	d
h	a	a	b	b	c	c	d	d

Итак, на множестве $\mathbb{Z}_8$ введены операции сложения и умножения.

1.4 Отрицательные числа и действия вычитания и деления на множестве $\mathbb{Z}_8^8$

Теперь необходимо определить действия вычитания и деления с помощью введенных операций. Но перед этим необходимо ввести отрицательные числа. Числа, меньшие нуля, то есть числа a , будут отрицательными, и если число отстоит на n мультипликативных единиц b от аддитивного нуля a в противоположную сторону от положительного направления, то оно будет равным $-\underbrace{(b + b + \dots + b)}_n$

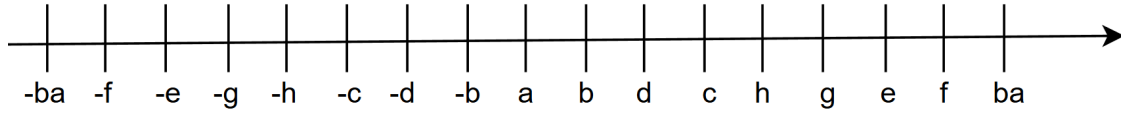


Рис. 2: геометрическая интерпретация $\mathbb{Z}_8^8$

Тогда минимально возможное число в рассматриваемом множестве $-ffffffffff$, а максимально возможное $ffffffffff$. Если при вычислении какой-либо операции получается число, лежащее вне отрезка $[-ffffffffff; fffffffffff]$, то данная операция не будет выполнена: произошло переполнение.

Вычитанием двух чисел Z_1 и Z_2 будет такое число Z_3 , что $Z_1 = Z_2 + Z_3$. Из рисунка видно, что, например, $ba - g = c$, $f - (-b) = ba$, $-e - (-c) = (-c)$. Действие вычитания связано с действием сложения, поэтому из реализации сложения немедленно следует реализация вычитания.

Если представить, что числовой ряд циклический с периодом в длину отрезка $[a; fffffffffff]$, то есть после числа $ffffffffff$ следует число a , и ввести понятие дополнения числа A — числа $\text{dor_}A$, такого, что $A + \text{dor_}A = fffffffffff + b = a$, то можно свести вычитание к сложению. Ниже изображена геометрическая интерпретация данного представления чисел из $\{x \in \mathbb{Z}_8^8 \mid a \leq x\}$ (рис. 3).

Из рисунка видно, что для того, чтобы найти разность между числами A и B , достаточно найти сумму A и $\text{dor_}B$ в условиях данных обозначений.

Умножение двух чисел подчиняется правилам умножения в $\mathbb{Z}$, а именно: произведение положительно, если каждый из множителей одного знака, и отрицателен в противном случае.

Частное двух чисел A и B — это такое число n , что $A = n * B$. Это верно в случае, если число A делится на число B нацело. В общем случае, неполным частным двух чисел A и B является число n такое, что $A = n * B + r$, r — остаток

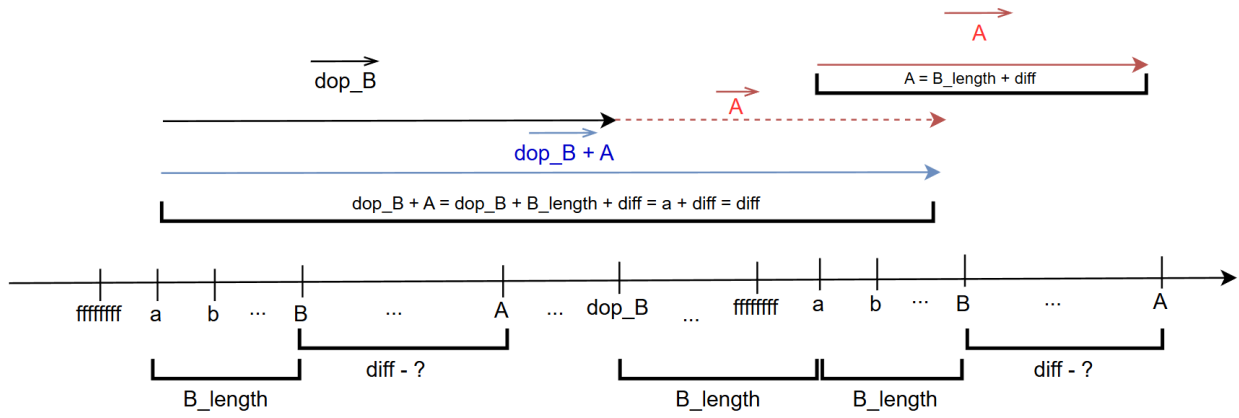


Рис. 3: числа на зацикленной прямой

от деления – неотрицательное число. Как и в обычной арифметике, $0 \leq r < B$.

Определено также деление на отрицательное число. Это действие так же подчиняется определениям выше.

Например,

$$ba / h = d, \quad h * d = ba; \quad ba / g = b[c], \quad ba = b * g + c;$$

$$-ba / c = -c[b], \quad -ba = -c * c + b; \quad -ba / (-e) = d[h], \quad -ba = d * (-e) + h$$

Также определены следующие действия:

$$\forall x \neq a : \quad x/a = \emptyset,$$

$$a/a = [-ffffff; fffffff].$$

Итак, были введены «малая» конечная арифметика и на ее основе «большая» конечная арифметика. Введенные арифметики являются кольцами $\langle \mathbb{Z}_8; +, * \rangle$ и $\langle \mathbb{Z}_8^8; +, * \rangle$ соответственно, так как выполнены все необходимые свойства кольца:

1. Сложение «+» ассоциативно.
2. Существует нейтральный элемент по операции сложения «+» – аддитивная единица a .
3. Для каждого элемента x из соответствующих множеств $\mathbb{Z}_8$ и $\mathbb{Z}_8^8$ существует обратный элемент по операции «+».
4. Сложение «+» коммутативно.
5. Умножение «*» ассоциативно.
6. Умножение «*» дистрибутивно относительно операции «+».

Более того, каждая из введенных арифметик является коммутативным кольцом с единицей, так как выполнены следующие свойства:

7. Умножение « $*$ » коммутативно.

8. Для каждого элемента x из соответствующих множеств $\mathbb{Z}_8$ и $\mathbb{Z}_8^8$ существует нейтральный элемент по операции « $*$ » – мультипликативная единица b .

2 Особенности реализации

Калькулятор «большой» конечной арифметики программно реализован с помощью нескольких классов на языке C++, описанных ниже. Была использована стандартная библиотека STL. Всего было написано 3 класса: 2 класса для реализации вычислений над $\langle \mathbb{Z}_8; +, * \rangle$ и 1 класс для реализации меню.

2.1 Класс Rules

Данный класс задает правила, по которым происходит сложение и умножение чисел. Эти классы реализуют логику таблиц сложения и умножения. Ниже приведен интерфейс класса (листинг 1).

```
1 class Rules{
2     private:
3         int bit_depth;
4         int cnt_syms;
5         char** table_plus = nullptr;
6
7         char** table_plus_c = nullptr;
8
9         char** table_times = nullptr;
10
11        char** table_times_c = nullptr;
12
13        std::vector<char> Hasse;
14
15        std::vector<std::string> plus;
16
17        std::vector<std::string> plus_c;
18
19        std::vector<std::string> times;
20
21        std::vector<std::string> times_c;
22
23
24    public:
25
26        Rules();
27
28        Rules(int, int);
29
30
31        void print_plus();
32        void print_plus_c();
33
```

```

34 void print_times();
35
36 void print_times_c();
37
38 void print_vector_plus();
39
40 char eval_plus(char one, char two) const;
41
42 char eval_plus_c(char one, char two) const;
43
44 char eval_times(char one, char two) const;
45
46 char eval_times_c(char one, char two) const;
47
48 std::vector<char> get_hasse();
49
50 };

```

Листинг 1: конструктор класса Rules

2.1.1 Конструктор Rules

Данный конструктор принимает на вход количество символов в описываемой «малой» арифметике и разрядность чисел в «большой» конечной арифметике. На выход подается объект типа Rules, с помощью которого можно обращаться к таблицам сложения и умножения (листинг 2).

Вход: `bit_depth` – количество символов в $\mathbb{Z}_8$, `cnt_syms` – количество символов в числе из множества $\mathbb{Z}_8^8$

Выход: инициализированный объект, конструктор которого был вызван

```

1 Rules::Rules(int bit_depth, int cnt_syms) : bit_depth(bit_depth), cnt_syms(
    cnt_syms), table_plus(nullptr),
2     table_plus_c(nullptr),
3     table_times(nullptr),
4     table_times_c(nullptr) {
5
6     Hasse.resize(8);
7     Hasse = {'a', 'b', 'd', 'c', 'h', 'g', 'e', 'f'};
8
9     plus.resize((bit_depth * (bit_depth-1) / 2) + bit_depth);
10
11     int k = 0;
12     std::string left_op = "a";
13     std::string right_op = "a";
14
15     while(k < 8){

```

```

16     char ar[9] = "abcdefgh";
17     right_op[0] = ar[k];
18
19     plus[k] = left_op + "+" + right_op + "=" + ar[k];
20
21     ++k;
22 }
23
24
25     plus[8] = "b+b=d";
26     plus[9] = "b+c=h";
27     plus[10] = "b+d=c";
28     plus[11] = "b+e=f";
29     plus[12] = "b+f=a";
30     plus[13] = "b+g=e";
31     plus[14] = "b+h=g";
32
33     plus[15] = "c+c=e";
34     plus[16] = "c+d=g";
35     plus[17] = "c+e=b";
36     plus[18] = "c+f=d";
37     plus[19] = "c+g=a";
38     plus[20] = "c+h=f";
39
40     plus[21] = "d+d=h";
41     plus[22] = "d+e=a";
42     plus[23] = "d+f=b";
43     plus[24] = "d+g=f";
44     plus[25] = "d+h=e";
45
46     plus[26] = "e+e=h";
47     plus[27] = "e+f=g";
48     plus[28] = "e+g=c";
49     plus[29] = "e+h=d";
50
51     plus[30] = "f+f=e";
52     plus[31] = "f+g=h";
53     plus[32] = "f+h=c";
54
55     plus[33] = "g+g=d";
56     plus[34] = "g+h=b";
57
58     plus[35] = "h+h=a";
59
60
61 // -----plus_c-----
62
63 plus_c.resize((bit_depth * (bit_depth-1) / 2) + bit_depth);

```

```

64
65 k = 0;
66 left_op = "a";
67 right_op = "a";
68
69 while(k < 8){
70     char ar[9] = "abcdefgh";
71     right_op[0] = ar[k];
72
73     plus_c[k] = left_op + "+" + right_op + "=" + "a";
74
75     ++k;
76 }
77
78
79 plus_c[8] = "b+b=a";
80 plus_c[9] = "b+c=a";
81 plus_c[10] = "b+d=a";
82 plus_c[11] = "b+e=a";
83 plus_c[12] = "b+f=b";
84 plus_c[13] = "b+g=a";
85 plus_c[14] = "b+h=a";
86
87 plus_c[15] = "c+c=a";
88 plus_c[16] = "c+d=a";
89 plus_c[17] = "c+e=b";
90 plus_c[18] = "c+f=b";
91 plus_c[19] = "c+g=b";
92 plus_c[20] = "c+h=a";
93
94
95 plus_c[21] = "d+d=a";
96 plus_c[22] = "d+e=b";
97 plus_c[23] = "d+f=b";
98 plus_c[24] = "d+g=a";
99 plus_c[25] = "d+h=a";
100
101 plus_c[26] = "e+e=b";
102 plus_c[27] = "e+f=b";
103 plus_c[28] = "e+g=b";
104 plus_c[29] = "e+h=b";
105
106 plus_c[30] = "f+f=b";
107 plus_c[31] = "f+g=b";
108 plus_c[32] = "f+h=b";
109
110 plus_c[33] = "g+g=b";
111 plus_c[34] = "g+h=b";

```

```

112
113     plus_c[35] = "h+h=b";
114
115
116 // -----TIMES-----
117
118     times.resize((bit_depth * (bit_depth-1) / 2) + bit_depth);
119     k = 0;
120     left_op = "a";
121     right_op = "a";
122
123     while(k < 8){
124         char ar[9] = "abcdefgh";
125         right_op[0] = ar[k];
126
127         times[k] = left_op + "+" + right_op + "=" + "a";
128
129         ++k;
130     }
131
132
133     times[8] = "b+b=b";
134     times[9] = "b+c=c";
135     times[10] = "b+d=d";
136     times[11] = "b+e=e";
137     times[12] = "b+f=f";
138     times[13] = "b+g=g";
139     times[14] = "b+h=h";
140
141     times[15] = "c+c=b";
142     times[16] = "c+d=e";
143     times[17] = "c+e=d";
144     times[18] = "c+f=g";
145     times[19] = "c+g=f";
146     times[20] = "c+h=h";
147
148
149     times[21] = "d+d=h";
150     times[22] = "d+e=h";
151     times[23] = "d+f=e";
152     times[24] = "d+g=d";
153     times[25] = "d+h=a";
154
155     times[26] = "e+e=h";
156     times[27] = "e+f=d";
157     times[28] = "e+g=e";
158     times[29] = "e+h=a";
159

```



```

160     times[30] = "f+f=b";
161     times[31] = "f+g=c";
162     times[32] = "f+h=h";
163
164     times[33] = "g+g=b";
165     times[34] = "g+h=h";
166
167     times[35] = "h+h=a";
168
169
170
171 // -----TIMES_C-----
172
173 times_c.resize((bit_depth * (bit_depth-1) / 2) + bit_depth);
174     k = 0;
175     left_op = "a";
176     right_op = "a";
177
178     while(k < 8){
179         char ar[9] = "abcdefgh";
180         right_op[0] = ar[k];
181
182         times_c[k] = left_op + "+" + right_op + "=" + "a";
183
184         ++k;
185     }
186
187
188     times_c[8] = "b+b=a";
189     times_c[9] = "b+c=a";
190     times_c[10] = "b+d=a";
191     times_c[11] = "b+e=a";
192     times_c[12] = "b+f=a";
193     times_c[13] = "b+g=a";
194     times_c[14] = "b+h=a";
195
196     times_c[15] = "c+c=b";
197     times_c[16] = "c+d=a";
198     times_c[17] = "c+e=d";
199     times_c[18] = "c+f=d";
200     times_c[19] = "c+g=b";
201     times_c[20] = "c+h=b";
202
203
204     times_c[21] = "d+d=a";
205     times_c[22] = "d+e=b";
206     times_c[23] = "d+f=b";
207     times_c[24] = "d+g=b";

```

```

208     times_c[25] = "d+h=b";
209
210     times_c[26] = "e+e=h";
211     times_c[27] = "e+f=g";
212     times_c[28] = "e+g=c";
213     times_c[29] = "e+h=c";
214
215     times_c[30] = "f+f=e";
216     times_c[31] = "f+g=h";
217     times_c[32] = "f+h=c";
218
219     times_c[33] = "g+g=c";
220     times_c[34] = "g+h=d";
221
222     times_c[35] = "h+h=a";
223
224 }
225

```

Листинг 2: конструктор класса Rules

Итак, теперь можно создать объект, содержащий в себе правила сложения и умножения. К нему можно обращаться при вычислениях.

2.1.2 Метод eval_plus

Метод eval_plus необходим для того, чтобы возвращать результат сложения чисел на $\mathbb{Z}_8$. Он работает непосредственно с построенным правилом сложения в конструкторе.

Вход: one – левый операнд сложения, two – правый операнд сложения.

Выход: str – результат сложения.

```

1 char Rules::eval_plus(char one, char two) const{
2     for(const auto& str: plus){
3         if((str[0] == one && str[2] == two) || (str[0] == two && str[2] == one)){
4             return str[4];
5         }
6     }
7     return 'q';
8 }

```

Листинг 3: вычисление сложения без учета переноса

Далее приведенные методы имеют аналогичную структуру и назначение. Они приведены ниже (листинги 4, 5, 6).

2.1.3 Метод eval_plus_c

Вход: one – левый операнд сложения, two – правый операнд сложения.

Выход: str – результат переноса при сложении.

```
1 char Rules::eval_plus_c(char one, char two) const{
2     for(const auto& str: plus_c){
3         if((str[0] == one && str[2] == two) || (str[0] == two && str[2] == one)){
4             return str[4];
5         }
6     }
7     return 'q';
8 }
```

Листинг 4: вычисление переноса при сложении

2.1.4 Метод eval_times

Вход: one – левый операнд умножения, two – правый операнд умножения.

Выход: str – результат умножения.

```
1 char Rules::eval_times(char one, char two) const{
2     for(const auto& str: times){
3         if((str[0] == one && str[2] == two) || (str[0] == two && str[2] == one)){
4             return str[4];
5         }
6     }
7     return 'q';
8 }
```

Листинг 5: вычисление умножения без учета переноса

2.1.5 Метод eval_times_c

Вход: one – левый операнд сложения, two – правый операнд умножения.

Выход: str – результат умножения.

```
1 char Rules::eval_times_c(char one, char two) const{
2     for(const auto& str: times_c){
3         if((str[0] == one && str[2] == two) || (str[0] == two && str[2] == one)){
4             return str[4];
5         }
6     }
7     return 'q';
8 }
```

Листинг 6: вычисление переноса при умножении

2.2 Класс Finit

Класс Finit реализует непосредственно числа из множества $\mathbb{Z}_8^8$, и действия над ними. Ниже приведена реализация интерфейса класса Finit (листинг 7)

```
1 class Finit{
2     private:
3
4         int bit_depth = 8;
5
6         std::string num1;
7
8         std::vector<char> aybuben;
9
10        Rules r;
11
12        std::string dop_num1;
13
14        bool sign = true;
15
16        bool lilbit = false;
17
18        bool full_segment = false;
19
20        bool empty_set = false;
21
22        bool rest = false;
23
24
25        std::string MIN{"-ffffffff"};
26        std::string MAX{"ffffffff"};
27
28    public:
29
30        Rules get_r();
31
32        Finit* rest_value = nullptr;
33
34        int translate();
35
36
37        Finit();
38
39        Finit(const Finit&);
40
41        Finit(std::string num1);
42
43        friend Finit operator+(const Finit& num1, const Finit& num2);
44
```

```

45     friend Finit operator-(const Finit& num1, Finit& num2);
46
47     friend Finit operator-(Finit num1);
48
49     Finit& operator=(const Finit&);
50
51     char operator[](int i) const;
52
53     friend bool operator<(const Finit& num1, const Finit& num2);
54
55     friend bool operator<=(const Finit&, const Finit&);
56
57     friend bool operator==(const Finit& num1, const Finit& num2);
58
59     friend std::ostream& operator<<(std::ostream& os, const Finit& obj);
60
61     void eval_dop();
62
63     std::string get_dop() const;
64
65     void make_normal_for_us(std::string&);
66
67     std::string make_normal_for_user(const std::string&) const;
68
69     std::string get_num1();
70
71     void set_lilbit(bool value);
72
73     bool get_lilbit() const;
74
75     bool get_sign() const;
76
77     void set_sign(bool flag);
78
79     int get_bit_depth() const;
80
81     int cnt_of_a() const;
82
83     void set_full_segment(bool flag);
84     void set_empty_set(bool flag);
85     void set_rest(bool flag);
86     bool get_rest() const;
87     bool get_empty_set()const;
88     bool get_full_segment()const;
89
90 };
91
92 Finit summ(Finit& obj1, Finit& obj2);

```

```

93
94 Finit subb(Finit& obj1, Finit& obj2);
95
96 Finit one_mult_one(Finit& obj1, Finit& obj2);
97
98 Finit mult_with_a(Finit& obj1, Finit& obj2);
99
100 Finit multy(Finit& obj1, Finit& obj2);
101
102 Finit int_div(Finit& obj1, Finit& obj2);
103
104 Finit gcd(Finit obj1, Finit obj2);

```

Листинг 7: интерфейс класса Finit

2.2.1 Конструктор класса Finit

Ниже приведена реализация конструктора класса Finit (листинг 8).

Вход: строка, представляющая число.

Выход: str – инициализированный объект класса Finit.

```

1 Finit::Finit(std::string num) : bit_depth(8) {
2     rest_value = new Finit;
3     r = Rules(8, 8);
4
5     if(num[0] == '-') {sign = false; num = num.substr(1);}
6     else sign = true;
7
8
9     make_normal_for_us(num);
10
11     this->num1 = num;
12
13     aybuben.resize(8);
14     aybuben = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'};
15 }

```

Листинг 8: конструктор класса Finit принимающий строку

2.2.2 Перегрузка оператора +

В данном классе также используется понятие дополнения числа, введенное в математическом описании. На основе него выполняются действия над числами: выполняется разность чисел, проверяется переполнение.

Ниже реализована глобальная перегрузка оператора + (листинг 9).

Вход: left_op – число из $\mathbb{Z}_8$, левый операнд сложения, right_op – число из $\mathbb{Z}_8$, правый операнд сложения.

Выход: str – «неверный» результат сложения.

```
1 Finit operator+(const Finit& left_op, const Finit& right_op){
2     Rules R(8, 8);
3
4     int len_left = left_op.num1.length();
5     int len_right = right_op.num1.length();
6
7     Finit ref_left_op(left_op);
8     Finit ref_right_op(right_op);
9
10    std::string res;
11    res.resize(8);
12
13    char x = 'a';
14    char x_new = 'a';
15
16    for(int i = 7; i != -1; --i){
17
18        char l = left_op.num1[i];
19        char r = right_op.num1[i];
20
21        res[i] = R.eval_plus(l, r);
22
23        x_new = R.eval_plus_c(l, r);
24
25        if(x_new == 'a') x_new = R.eval_plus_c(res[i], x);
26
27        res[i] = R.eval_plus(res[i], x);
28
29
30        x = x_new;
31    }
32
33    Finit obj(res);
34    return obj;
35 }
```

Листинг 9: неправильное сложение двух чисел

Эта операция возвращает «неверный» результат сложения двух чисел, однако играет большую роль в курсовой работе. Результат неверный, потому что сумма двух чисел находится в смысле представления чисел из рис. 3, что является лишь вспомогательным и не имеет смысла в условиях исходной задачи.

2.2.3 Метод eval_dop

Ниже приведена реализация метода eval_dop, (листинг 10).

Вход: dop_num_1 – неинициализированное дополнение числа до максимально возможного.

Выход: вычисленное дополнение числа.

```
1 void Finit::eval_dop(){
2     Rules R(bit_depth, bit_depth);
3     std::string last_num("f", bit_depth);
4
5     std::string almost_dop;
6
7     almost_dop.resize(bit_depth);
8
9     for(int i = 0; i < bit_depth; ++i){
10         for(const auto& el: aybuben){
11             if(R.eval_plus(num1[i], el) == 'f') almost_dop[i] = el;
12         }
13     }
14     std::string one_str = "aaaaaaab";
15     Finit one(one_str);
16
17     Finit cur(almost_dop);
18
19     this->dop_num1 = (cur + one).num1;
20
21     if(num1 == "aaaaaaaa") this->dop_num1 = "ffffffff";
22
23 }
```

Листинг 10: вычисление дополнения числа

2.2.4 Перегрузка операторов <, ==, <=

Ниже приведена перегрузка оператора < (листинг 11).

Вход: num1 и num2 – строки, числа из $\mathbb{Z}_8^8$.

Выход: flag – булево значение, результат проверки на то, что число num1 < числа num2.

```
1 bool operator<(const Finit& num1, const Finit& num2){
2     int bit_depth = num1.bit_depth;
3
4     Rules r = num1.r;
5 }
```



```

6      std::vector<char> hasse = r.get_hasse();
7      bool flag = false;
8
9      for(int i = 0; i < bit_depth; ++i){
10         auto it = std::find(hasse.begin(), hasse.end(), num1[i]);
11         int index_1 = std::distance(hasse.begin(), it);
12         auto it1 = std::find(hasse.begin(), hasse.end(), num2[i]);
13         int index_2 = std::distance(hasse.begin(), it1);
14
15         if(index_1 < index_2) {flag = true; break;}
16         else if(index_1 > index_2) {flag = false; break;}
17     }
18
19     return flag;
20 }

```

Листинг 11: перегрузка оператора <

Ниже приведены перегрузки операторов == и <=, реализации которых основаны на перегрузке оператора < (листинги 12, 13).

Вход: left_op – число из $\mathbb{Z}_8^8$, левый операнд сравнения, right_op – число из $\mathbb{Z}_8^8$, правый операнд сравнения.

Выход: flag – установленный флаг, сигнализирующий о равенстве или неравенстве между числами

```

1 bool operator==(const Finit& left_op, const Finit& right_op){
2     int bit_depth = left_op.bit_depth;
3
4     for(int i = 0; i < bit_depth; ++i){
5         if(left_op[i] != right_op[i]) return false;
6     }
7     return true;
8 }

```

Листинг 12: оператор сравнения

Вход: left_op – число из $\mathbb{Z}_8^8$, левый операнд сравнения, right_op – число из $\mathbb{Z}_8^8$, правый операнд сравнения.

Выход: flag – установленный флаг, сигнализирующий о том, что левый операнд не больше правого.

```

1 bool operator<=(const Finit& obj1, const Finit& obj2){
2     if(obj1 < obj2 || obj1 == obj2) return true;
3     return false;
4 }

```

Листинг 13: перегрузка оператора <

Данные операторы играют не менее важную роль и применяется во многих функциях. Стоит отметить, что они сравнивают лишь неотрицательные числа,

то есть они могут быть применимы для сравнения модулей чисел. В общем случае его применение приведет к ошибкам.

При каждом действии с числами необходимо учитывать их знак: при сравнении, сложении, умножении, и так далее. Для этого хранится специальный флаг в классе `sign`, который показывает знак числа. Ниже приведены функции `get'er` и `set'er` для управления этим полем (листинги 14, 15).

Вход: `flag` – булево значение, которое необходимо установить как знак числа

Выход: `flag` – установленный флаг в объекте.

```
1 bool Finit::get_sign() const{return sign;}
```

Листинг 14: получение знака числа

Вход: `flag` – булево значение, которое необходимо установить как знак числа

Выход: `flag` – установленный флаг в объекте.

```
1 void Finit::set_sign(bool flag){sign = flag;}
```

Листинг 15: установка знака числа

2.2.5 Перегрузка оператора —

Ниже реализована функция, находящая «неверную» разность двух чисел (листинг 16).

Вход: `left_op` – число из $\mathbb{Z}_8^8$, левый операнд вычитания, `right_op` – число из $\mathbb{Z}_8^8$, правый операнд вычитания.

Выход: `str` – «неверный» результат вычитания.

```
1 Finit operator-(const Finit& left_op, Finit& right_op){
2     Rules R(8, 8);
3     right_op.eval_dop();
4     Finit tmp(right_op.get_dop());
5     return left_op + tmp;
6 }
```

Листинг 16: разность двух чисел

Перегруженный оператор — аналогично оператору `+` возвращает неверный результат, так как он умеет вычитать только из большего числа меньшее число. Это частный случай вычитания, и не может быть пригодным для вычисления действия вычитания, однако может быть частью его реализации, так как все случаи вычитания сводятся к вычитанию из большего числа меньшего, с точностью до знака.

2.2.6 Сумма двух чисел

Ниже реализована функция, находящая верную сумму двух чисел (листинг 17).

Вход: obj1, obj2 – два числа из $\mathbb{Z}_8$.

Выход: sum – сумма чисел obj1 и obj2.

```
1 Finit summ(Finit& obj1, Finit& obj2){
2     if(obj1.get_sign() && obj2.get_sign()){
3         Finit sum = obj1 + obj2;
4         sum.set_sign(true); // positive sum
5         obj1.eval_dop();
6         obj2.eval_dop();
7         Finit obj1_dop = obj1.get_dop();
8         Finit obj2_dop = obj2.get_dop();
9         if(obj1_dop <= obj2 || obj2_dop <= obj1) sum.set_lilbit(true);
10        return sum;
11    }
12    else if(!obj1.get_sign() && !obj2.get_sign()){
13        Finit sum = obj1 + obj2;
14        sum.set_sign(false); // negative sum
15        obj1.eval_dop();
16        obj2.eval_dop();
17        Finit obj1_dop = obj1.get_dop();
18        Finit obj2_dop = obj2.get_dop();
19        if(obj1_dop <= obj2 || obj2_dop <= obj1) sum.set_lilbit(true);
20        return sum;
21    }
22    else if(obj1.get_sign() && !obj2.get_sign()){
23        Finit sum;
24        if(obj1 < obj2){sum = obj2 - obj1; sum.set_sign(false);}
25        else{sum = obj1 - obj2; sum.set_sign(true);}
26        return sum;
27    }
28    else if(!obj1.get_sign() && obj2.get_sign()){
29        Finit sum;
30        if(obj1 < obj2){sum = obj2 - obj1; sum.set_sign(true);}
31        else{sum = obj1 - obj2; sum.set_sign(false);}
32        return sum;
33    }
34    return Finit();
35 }
```

Листинг 17: сумма двух чисел

В данной функции находится корректная сумма двух чисел. Рассматриваются случаи различных знаков операндов. Таким образом, частично правильно работающие функции: операторы сравнения, оператор +, оператор −, пригодились

для реализации верно работающей функции, находящей сумму двух чисел.

2.2.7 Разность двух чисел

Ниже реализована функция, находящая верную разность двух чисел (листинг 18).

Вход: `obj1`, `obj2` – два числа из $\mathbb{Z}_8^*$.

Выход: `sum` – разность чисел `obj1` и `obj2`.

```
1 Finit subb(Finit& obj1, Finit& obj2){
2     Finit tmp = obj2;
3     if(obj2.get_sign()) tmp.set_sign(false);
4     else tmp.set_sign(true);
5     return summ(obj1, tmp);
6 }
```

Листинг 18: разность двух чисел

Данная функция имеет несложную реализацию, так как основывается на реализации суммы двух чисел: разность двух чисел есть сумма первого числа и противоположного второго.

2.2.8 Произведение двух чисел

Ниже реализована функция, находящая произведение двух чисел (листинг 19).

Вход: `flag` – `obj1`, `obj2` – два числа из $\mathbb{Z}_8^*$.

Выход: `ans` – произведение чисел `obj1` и `obj2`.

```
1 Finit multy(Finit& obj1, Finit& obj2){
2     if(obj1.get_num1() == "a" || obj2.get_num1() == "a") return Finit("a");
3     Finit cnt("ab");
4     Finit step("ab");
5
6     Finit min, max;
7     min = (obj1 < obj2) ? obj1 : obj2;
8     max = (obj1 < obj2) ? obj2 : obj1;
9     Finit ans(max);
10
11     if(ans.get_lilbit()) std::cout << "error!\n";
12     while(cnt < min){
13         Finit sum(summ(ans, max));
14         ans = sum;
15         cnt = summ(step, cnt);
16     }
```

```

16         if(ans.get_lilbit()) break;
17     }
18
19     if(obj1.get_sign() == obj2.get_sign()) ans.set_sign(true);
20     else ans.set_sign(false);
21
22
23     return ans;
24
25 }

```

Листинг 19: произведение двух чисел

Функция нахождения произведения реализована на основе сложения: любое произведение можно представить в виде суммы некоторого количества одинаковых слагаемых. Знак результата учитывается, переполнение так же отслеживается.

2.2.9 Деление двух чисел

Ниже реализована функция, находящая (неполное)частное двух чисел (листинг 20).

Вход: flag – obj1, obj2 – два числа из $\mathbb{Z}_8$.

Выход: res – (неполное)частное чисел obj1 и obj2, и остаток r, если не он не равен нулю.

```

1 Finit int_div(Finit& obj1, Finit& obj2){
2     Finit e("a");
3     Finit add("b");
4     Finit res("a");
5
6     if(obj1 == res && obj2 == res) {res.set_full_segment(true); return res;}
7     if(obj2 == res) {res.set_empty_set(true); return res;}
8
9
10    if(e < obj1 && e < obj2 && (obj1.get_sign() && obj2.get_sign()
11        ) ){
12
13        if(obj1 < obj2) {res.set_rest(true); *res.rest_value = obj1; return res;}
14        Finit tmp(obj1);
15        while(obj2 <= tmp){
16            tmp = subb(tmp, obj2);
17            res = summ(res, add);
18        }
19        if(e < tmp){
20            res.set_rest(true);

```

```

21         *res.rest_value = tmp;
22     }
23     (*res.rest_value).set_sign(true);
24 }
25 else if(e < obj1 && e < obj2 && !obj1.get_sign() && obj2.get_sign()){
26     Finit tmp(obj1);
27     while(!tmp.get_sign()){
28         if(tmp <= obj2 && !tmp.get_sign()){tmp.set_sign(true); tmp = subb(
obj2, tmp);}
29         else{tmp = summ(tmp, obj2);}
30         res = summ(res, add);
31     }
32     if(tmp.get_sign() && (e < tmp)){
33         res.set_rest(true);
34         *res.rest_value = tmp;
35     }
36     res.set_sign(false);
37
38 }
39
40 else if(e < obj1 && e < obj2 && !obj1.get_sign() && !obj2.get_sign()){
41     Finit tmp(obj1);
42     Finit obj2_2(-obj2);
43     obj2_2.set_sign(true);
44     while(!tmp.get_sign()){
45         if(tmp <= obj2_2 && !tmp.get_sign()){tmp.set_sign(true); tmp = subb(
obj2_2, tmp);}
46         else{tmp = summ(tmp, obj2_2);}
47         res = summ(res, add);
48     }
49     if(tmp.get_sign() && (e < tmp)){
50         res.set_rest(true);
51         *res.rest_value = tmp;
52     }
53     res.set_sign(true);
54 }
55
56 else if(e < obj1 && e < obj2 && (obj1.get_sign() && !obj2.get_sign()
57         ) ){
58     if(obj1 < obj2) {res.set_rest(true); res.set_sign(true); *res.rest_value
= obj1; return res;}
59     Finit obj2_obj2(obj2);
60     obj2_obj2.set_sign(true);
61     Finit tmp(obj1);
62     while(obj2_obj2 <= tmp){
63         tmp = subb(tmp, obj2_obj2);
64         res = summ(res, add);
65     }

```

```

66         if(e < tmp){
67             res.set_rest(true);
68             *res.rest_value = tmp;
69         }
70         (*res.rest_value).set_sign(true);
71         res.set_sign(false);
72     }
73
74     return res;
75 }

```

Листинг 20: деление двух чисел

Функция нахождения (неполного) частного реализована на основе вычитания: любое деление можно представить в виде вычитания некоторого количества одинаковых чисел, делителей. Знак результата учитывается и высчитывается остаток. Делить можно числа любого знака на числа любого знака. Также предусмотрены случаи деления ненулевого числа на нуль и деления нуля на нуль: в первом случае выводится значок пустого множества, во втором случае – сообщение, что результат равен отрезку от минимального числа до максимального.

Ниже приведены реализации вспомогательных методов при реализации действия деления (листинги [21](#), [22](#), [23](#))

Вход: flag – булевой значение для установки истинности целочисленного деления

Выход: инициализированный флаг rest внутри класса

```

1 void Finit::set_rest(bool flag){rest = flag;}

```

Листинг 21: функция возвращающая истинность целочисленного деления

Вход: flag – булевой значение для установки истинности деления a/a

Выход: инициализированный флаг full_segment внутри класса

```

1 void Finit::set_full_segment(bool flag){full_segment = flag;}

```

Листинг 22: функция возвращающая истинность деления a/a

Вход: flag – булевой значение для установки истинности целочисленного деления на a

Выход: инициализированный флаг empty_set внутри класса

```

1 void Finit::set_empty_set(bool flag){empty_set = flag;}

```

Листинг 23: функция возвращающая истинность деления на a

2.2.10 Вывод числа на консоль

Ниже приведена реализация перегрузки оператора «», необходимого для вывода числа или результата действия над ним на консоль (листинг 24).

Вход: m – ссылка на поток вывода, obj – объект, число, которое необходимо вывести

Выход: ссылка на поток вывода.

```
1 std::ostream& operator<<(std::ostream& m, const Finit& obj) {
2     std::string ans = obj.make_normal_for_user(obj.num1);
3     if(obj.lilbit){ m << "perepolnenie!"; return m;}
4     if(obj.get_empty_set()) {m << "Empty set"; return m;}
5     if(obj.get_full_segment()) {m << "[" << obj.MIN << "; " << obj.MAX << "];"
6     return m;}
7     if(!obj.get_sign()) m << "-";
8     if(obj.get_rest()) {m << ans << "[" << *obj.rest_value << "];" return m;}
9     m << ans;
10    return m;
11 }
```

Листинг 24: перегруженный оператор вывода

В данной функции используется еще одна вспомогательная функция – `make_normal_for_user`, которая переводит число в форму, удобную для пользователя. В классе `Finit` числа представлены в виде восьми разрядов, с включением незначащих. Это форма удобна для вычислений внутри методов класса, но пользователю такое представление ни к чему, поэтому необходимо отбросить незначащие разряды. Ниже приведена реализация функции `make_normal_for_user` (листинг 25).

Вход: num – строка, число из $\mathbb{Z}_8^8$

Выход: ans – строка, не содержащая незначащие разряды.

```
1 std::string Finit::make_normal_for_user(const std::string& num) const{
2     std::string ans;
3     ans.resize(10);
4     for(int i = 0; i < bit_depth; ++i){
5         if(num[i] != 'a'){
6             ans = num.substr(i);
7             break;
8         }
9     }
10    if(num == "aaaaaaaa") return "a";
11    return ans;
12 }
```

Листинг 25: представление числа без незначащих разрядов

3 Результаты работы программы

При запуске программы выводится консольное меню (рис. 4)

```
arkady@DESKTOP-21RRVR5:~/discra_kursach$ g++ main.cpp Rules.cpp Finit.cpp menu.cpp checks.cpp
arkady@DESKTOP-21RRVR5:~/discra_kursach$ ./a.out
Hello, world!

-----
| Курсовая работа. Калькулятор <<большой>> конечной арифметики |
|-----|
|                                     |
|             Menu                   |
|                                     |
|-----|

Диаграмма Хассе: a -- b -- d -- c -- h -- g -- e -- f

Выберите действие:
=====
|                               |
| Выберите действие из списка ниже: |
|                               |
| [1] > Сложение                |
| [2] > Вычитание                |
| [3] > Умножение                |
| [4] > Деление                  |
| [5] > Вывод таблицы сложения   |
| [6] > Вывод таблицы сложения с переносом |
| [7] > Вывод таблицы умножения  |
| [8] > Вывод таблицы умножения с переносом |
| [9] > Выход                    |
|=====|
```

Рис. 4: консольное меню

Предлагается выбрать действие. Если пользователь ввел некорректные данные, программа запросит ввести данные еще раз (рис. 5, 6, 7).

```
0
Некорректный ввод! Попробуйте ввести данные еще раз!
```

Рис. 5: ввод некорректных данных в меню, 1

```
gr
Некорректный ввод! Попробуйте выбрать действие еще раз!
```

Рис. 6: ввод некорректных данных в меню, 2

```
\h
Некорректный ввод! Попробуйте выбрать действие еще раз!
```

Рис. 7: ввод некорректных данных в меню, 3

Пользователь может вывести таблицы сложения и умножения в «малой» конечной арифметике (рис. 8, 9, 10, 11).

```
5
Ваше действие -- [5]
---Table of addition +_s---
+s| a  b      c      d      e      f      g      h
-----
a | a  b      c      d      e      f      g      h
b | b  d      h      c      f      a      e      g
c | c  h      e      g      b      d      a      f
d | d  c      g      h      a      b      f      e
e | e  f      b      a      h      g      c      d
f | f  a      d      b      g      e      h      c
g | g  e      a      f      c      h      d      b
h | h  g      f      e      d      c      b      a

      |-----|
      | Menu |
      |-----|
```

Рис. 8: вывод таблицы сложения

```
6
Ваше действие -- [6]
---Table of addition +_c---
+c| a  b      c      d      e      f      g      h
-----
a | a  a      a      a      a      a      a      a
b | a  a      a      a      a      a      b      a
c | a  a      a      b      b      b      a      a
d | a  a      a      a      b      b      a      a
e | a  a      b      b      b      b      b      b
f | a  b      b      b      b      b      b      b
g | a  a      b      a      b      b      b      b
h | a  a      a      a      b      b      b      b

      |-----|
      | Menu |
      |-----|
```

Рис. 9: вывод таблицы сложения с переносом

```

7
Ваше действие -- [7]

---Table of multiplication *_s---

*s| a  b      c      d      e      f      g      h
---
a | a  a      a      a      a      a      a      a
b | a  b      c      d      e      f      g      h
c | a  c      b      e      d      g      f      h
d | a  d      e      h      h      e      d      a
e | a  e      d      h      h      d      e      a
f | a  f      g      e      d      b      c      h
g | a  g      f      d      e      c      b      h
h | a  h      h      a      a      h      h      a

          |-----|
          | Menu |
          |-----|

```

Рис. 10: вывод таблицы умножения

```

8
---Table of multiplication *_c---

*c| a  b      c      d      e      f      g      h
---
a | a  a      a      a      a      a      a      a
b | a  a      a      a      a      a      a      a
c | a  a      b      a      d      d      b      b
d | a  a      a      a      b      b      b      b
e | a  a      d      b      h      g      c      c
f | a  a      d      b      g      e      h      c
g | a  a      b      b      c      h      c      d
h | a  a      b      b      c      c      d      d

          |-----|
          | Menu |
          |-----|

```

Рис. 11: вывод таблицы умножения с переносом

Итак, теперь пусть пользователь введет действие [1]. Ему будет предложено ввести первое слагаемое. Если ввести некорректные данные, а именно цифры, не принадлежащие носителю данной в условии арифметики $\langle \mathbb{Z}_8, +, * \rangle$, или числа, не принадлежащие носителю $\langle \mathbb{Z}_8^8, +, * \rangle$. Тогда произойдет следующее (рис. 12).

Итак, вводимые данные проверяются на корректность. Пусть теперь пользователь вводит корректные значения. Ниже проиллюстрированы некоторые примеры сложения чисел в $\langle \mathbb{Z}_8, +, * \rangle$ (рис. 13, 14).

```
1
Ваше действие -- [1]
Введите первое число из диапазона [-ffffffff; ffffffffff]:
rt
Введите корректные данные!
tg
Введите корректные данные!
we
Введите корректные данные!
ds
Введите корректные данные!
02
Введите корректные данные!
bbbbbbbbbbbbbbbbbb
Введите корректные данные!
```

Рис. 12: некорректный ввод при сложении

```
ds
Введите корректные данные!
02
Введите корректные данные!
a
Введите второе число из диапазона [-ffffff; fffffff]:
g
Итак вы ввели два числа:

Первое число: a
Второе число: g
```

| nnnnnnnnnnnnnn |
| a + g = g |
| nnnnnnnnnnnnnn |

Рис. 13: ввод простых чисел, 1

```

Введите первое число из диапазона [-ffffffff; ffffffffff]:
b
Введите второе число из диапазона [-ffffffff; ffffffffff]:
b
Итак вы ввели два числа:

Первое число: b
Второе число: b

Menu

| ~~~~~ |
| b + b = d |
| ~~~~~ |

```

Рис. 14: ввод простых чисел, 2

Теперь пусть пользователь попробует ввести не совсем простейшие данные. Для начала, если он введет составные, но не столь большие числа, то сумма будет найдена, согласно таблицам сложения (рис. 15)

```

Введите первое число из диапазона [-ffffffff; ffffffffff]:
bdg
Введите второе число из диапазона [-ffffffff; ffffffffff]:
dfe
Итак вы ввели два числа:

Первое число: bdg
Второе число: dfe

Menu

| ~~~~~ |
| bdg + dfe = hdc |
| ~~~~~ |

```

Рис. 15: введение «непростых» чисел

Если пользователь введет большие числа, и попытается их сложить, то при допустимом результате он выведется, при переполнении выведется соответствующее сообщение (рис. 16, 17, 18, 19)

```

Введите первое число из диапазона [-ffffffff; ffffffffff]:
gggggggg
Введите второе число из диапазона [-ffffffff; ffffffffff]:
bgfefef
Итак вы ввели два числа:

Первое число: gggggggg
Второе число: bgfefef

Menu

| ~~~~~ |
| gggggggg + bgfefef = gfcghghh |
| ~~~~~ |

```

Рис. 16: Ввод больших чисел, 1

```

Введите первое число из диапазона [-ffffffff; fffffffff]:
ffffffe
Введите второе число из диапазона [-ffffffff; fffffffff]:
b
Итак вы ввели два числа:

Первое число: fffffffe
Второе число: b

|-----|
| Menu |
|-----|

|-----|
| fffffffe + b = ffffffff |
|-----|

```

Рис. 17: ввод больших чисел, 2

```

Введите первое число из диапазона [-ffffffff; fffffffff]:
fffaabbdс
Введите корректные данные!
fffaabdf
Введите второе число из диапазона [-ffffffff; fffffffff]:
cccdfeee
Итак вы ввели два числа:

Первое число: fffaabdf
Второе число: cccdfeee

|-----|
| Menu |
|-----|

|-----|
| fffaabdf + cccdfeee = Переполнение! |
|-----|

```

Рис. 18: получение переполнения, 1

```

Введите первое число из диапазона [-ffffffff; fffffffff]:
ffffffff
Введите второе число из диапазона [-ffffffff; fffffffff]:
b
Итак вы ввели два числа:

Первое число: fffffffff
Второе число: b

|-----|
| Menu |
|-----|

|-----|
| fffffffff + b = Переполнение! |
|-----|

```

Рис. 19: получение переполнения, 2

Далее, если пользователь введет отрицательные числа, и попытается их сложить, то все будет работать корректно (рис. 20, 21, 22).

```

Введите первое число из диапазона [-ffffffff; ffffffffff]:
a
Введите второе число из диапазона [-ffffffff; ffffffffff]:
-gd
Итак вы ввели два числа:

Первое число: a
Второе число: -gd

Menu

|-----|
| a + -gd = -gd |
|-----|

```

Рис. 20: ввод отрицательных чисел, 1

```

Введите первое число из диапазона [-ffffffff; ffffffffff]:
-ffffffff
Введите второе число из диапазона [-ffffffff; ffffffffff]:
ffffffff
Итак вы ввели два числа:

Первое число: -ffffffff
Второе число: fffffffff

Menu

|-----|
| -ffffffff + fffffffff = a |
|-----|

```

Рис. 21: ввод отрицательных чисел, 2

```

Введите первое число из диапазона [-ffffffff; ffffffffff]:
-fffffffe
Введите второе число из диапазона [-ffffffff; ffffffffff]:
-bd
Итак вы ввели два числа:

Первое число: -fffffffe
Второе число: -bd

Menu

|-----|
| -fffffffe + -bd = Переполнение! |
|-----|

```

Рис. 22: ввод отрицательных чисел, 3

Теперь, пусть пользователь введет действие [2]. Фактически, уже были рассмотрены случаи с действием вычитания, так как при вводе отрицательных чисел и последующем их сложении происходит действие вычитания. Однако вычитание должно быть отдельным действием, поэтому оно реализовано в меню, и ниже прикреплены [23](#), [24](#), [25](#), [26](#), на которых изображено действие вычитания в калькуляторе, а на последнем рисунке показано, что отрицательные числа также поддерживаются.

```

Введите первое число из диапазона [-ffffffff; fffffffff]:
a
Введите второе число из диапазона [-ffffffff; fffffffff]:
a
Итак вы ввели два числа:

Первое число: a
Второе число: a

Menu

| ~~~~~ |
| a - a = a |
| ~~~~~ |

```

Рис. 23: вычитание чисел, 1

```

Введите первое число из диапазона [-ffffffff; fffffffff]:
bd
Введите второе число из диапазона [-ffffffff; fffffffff]:
f
Итак вы ввели два числа:

Первое число: bd
Второе число: f

Menu

| ~~~~~ |
| bd - f = c |
| ~~~~~ |

```

Рис. 24: вычитание чисел, 2

```

Введите первое число из диапазона [-ffffffff; fffffffff]:
f
Введите второе число из диапазона [-ffffffff; fffffffff]:
bd
Итак вы ввели два числа:

Первое число: f
Второе число: bd

Menu

| ~~~~~ |
| f - bd = -c |
| ~~~~~ |

```

Рис. 25: вычитание чисел, 3

```

Введите первое число из диапазона [-ffffffff; fffffffff]:
ffffffff
Введите второе число из диапазона [-ffffffff; fffffffff]:
-b
Итак вы ввели два числа:

Первое число: ffffffff
Второе число: -b

Menu

| ~~~~~ |
| ffffffff - -b = Переполнение! |
| ~~~~~ |

```

Рис. 26: вычитание чисел, 4

Итак, пусть пользователь ввел действие [3]. Ниже изображены результаты работы программы при различных данных (рис. 27, 28, 29, 30, 31).

```

Введите первое число из диапазона [-ffffffff; fffffffff]:
bgdf
Введите второе число из диапазона [-ffffffff; fffffffff]:
a
Итак вы ввели два числа:

Первое число: bgdf
Второе число: a

|-----|
| Menu |
|-----|

|-----|
| bgdf * a = a |
|-----|

```

Рис. 27: умножение на a

```

Введите первое число из диапазона [-ffffffff; fffffffff]:
-c
Введите второе число из диапазона [-ffffffff; fffffffff]:
h
Итак вы ввели два числа:

Первое число: -c
Второе число: h

|-----|
| Menu |
|-----|

|-----|
| -c * h = -bh |
|-----|

```

Рис. 28: умножение отрицательного на положительное

```

Введите первое число из диапазона [-ffffffff; fffffffff]:
bffe
Введите второе число из диапазона [-ffffffff; fffffffff]:
-fe
Итак вы ввели два числа:

Первое число: bffe
Второе число: -fe

|-----|
| Menu |
|-----|

|-----|
| bffe * -fe = -bfceah |
|-----|

```

Рис. 29: умножение положительного на отрицательное


```

Введите первое число из диапазона [-ffffff; fffffff]:
-bd
Введите второе число из диапазона [-ffffff; fffffff]:
g
Итак вы ввели два числа:

Первое число: -bd
Второе число: g

|-----|
|          Menu          |
|-----|

|-----|
| -bd / g = -d          |
|-----|

```

Рис. 33: целочисленное деление отрицательного на положительное

```

Введите первое число из диапазона [-ffffff; fffffff]:
da
Введите второе число из диапазона [-ffffff; fffffff]:
-h
Итак вы ввели два числа:

Первое число: da
Второе число: -h

|-----|
|          da / -h = -h  |
|-----|

```

Рис. 34: целочисленное деление положительного на отрицательное

```

Введите первое число из диапазона [-ffffff; fffffff]:
-bbbb
Введите второе число из диапазона [-ffffff; fffffff]:
-g
Итак вы ввели два числа:

Первое число: -bbbb
Второе число: -g

|-----|
|          -bbbb / -g = bbg |
|-----|

|-----|
|          Menu          |
|-----|

```

Рис. 35: целочисленное деление отрицательного на отрицательное

```

Введите первое число из диапазона [-ffffff; fffffff]:
bc
Введите второе число из диапазона [-ffffff; fffffff]:
c
Итак вы ввели два числа:

Первое число: bc
Второе число: c

|-----|
|          bc / c = c[d]  |
|-----|

|-----|
|          Menu          |
|-----|

```

Рис. 36: деление положительных чисел с остатком

```

Введите первое число из диапазона [-ffffff; fffffff]:
-bc
Введите второе число из диапазона [-ffffff; fffffff]:
c
Итак вы ввели два числа:

Первое число: -bc
Второе число: c

| ~~~~~ |
| -bc / c = -h[b] |
| ~~~~~ |

```

Рис. 37: деление отрицательного числа на положительное с остатком

```

Введите первое число из диапазона [-ffffff; fffffff]:
bc
Введите второе число из диапазона [-ffffff; fffffff]:
-c
Итак вы ввели два числа:

Первое число: bc
Второе число: -c

| ~~~~~ |
| bc / -c = -c[d] |
| ~~~~~ |

|-----|
|      Menu      |
|-----|

```

Рис. 38: Деление положительного числа на отрицательное с остатком

```

Введите первое число из диапазона [-ffffff; fffffff]:
-fdaeegfe
Введите второе число из диапазона [-ffffff; fffffff]:
-fdbea
Итак вы ввели два числа:

Первое число: -fdaeegfe
Второе число: -fdbea

| ~~~~~ |
| -fdaeegfe / -fdbea = baaa[fbdad] |
| ~~~~~ |

|-----|
|      Menu      |
|-----|

```

Рис. 39: деление отрицательного числа на отрицательное

Пусть пользователь попытался поделить на a . Результат будет следующим (рис. 40).

```
Введите первое число из диапазона [-ffffff; fffffff]:
bgdf
Введите второе число из диапазона [-ffffff; fffffff]:
a
Итак вы ввели два числа:

Первое число: bgdf
Второе число: a

|-----|
| Menu |
|-----|

|-----|
| bgdf / a = ∅ |
|-----|
```

Рис. 40: деление на a

Наконец, пусть пользователь попытался поделить a на a . Тогда результат будет следующим (рис. 41)

```
Введите первое число из диапазона [-ffffff; fffffff]:
aaaaaaaaaa
Введите второе число из диапазона [-ffffff; fffffff]:
aaaaaaaaaaaaaa
Итак вы ввели два числа:

Первое число: a
Второе число: a

|-----|
| Menu |
|-----|

|-----|
| a / a = [-ffffff; fffffff] |
|-----|
```

Рис. 41: деление a/a

Итак, пользователь может закончить работу с калькулятор, введя [9]. Программа завершится (рис. 42).

```

| ~~~~~
| ffffffff / ffff = baaab
| ~~~~~

```

Диаграмма Хассе: $a \dashv b \dashv d \dashv c \dashv h \dashv g \dashv e \dashv f$

```
=====
                          Выберите действие из списка ниже:
[1] > Сложение
[2] > Вычитание
[3] > Умножение
[4] > Деление
[5] > Вывод таблицы сложения
[6] > Вывод таблицы сложения с переносом
[7] > Вывод таблицы умножения
[8] > Вывод таблицы умножения с переносом
[9] > Выход
=====
```

```
arkady@DESKTOP-21RRVR5:~/discra_kursach$
```

46

Заключение

В ходе лабораторной работы был сформирован калькулятор «большой» конечной арифметики, реализующий действия над числами из $\langle \mathbb{Z}_8; +, * \rangle$. Для этого были определены операции сложения $+$ и умножения $*$ в «малой» конечной арифметике $\langle \mathbb{Z}_8; +, * \rangle$ исходя из заданного «правила $+1$ ». Далее «малая» конечная арифметика была расширена до «большой», и в ней уже было задано отношение порядка, введены отрицательные числа и определены действия умножения и деления, как целочисленного, так и с остатком.

После определения конечных арифметик и действий над ними «большая» конечная арифметика была реализована программно в виде калькулятора. Калькулятор может выводить таблицы сложения, умножения и вычислять результаты действий сложения, вычитания, умножения и деления над числами из $\langle \mathbb{Z}_8; +, * \rangle$. Для этого было реализовано пользовательское меню, и была предусмотрена обработка некорректного ввода.

К достоинству реализации стоит отнести отказ от использования «сырых» указателей (в реализации используется лишь один «сырой» указатель, и тот является «индикатором» деления с остатком, то есть не используется для каких-либо непростых действий по типу копирования, удаления и присваивания нового адреса из кучи). Их использование усложнило бы структуру кода и ухудшило его читабельность.

Также использование одних функций, или действий, для реализации других функций, или действий, уменьшает объем кода и упрощает его логику. Например, вычитание почти полностью реализовано через сложение: нужно лишь поменять знак вычитателя на противоположный и сложить вычитаемое и получившееся слагаемое.

К недостаткам реализации важно отнести способ хранения таблиц сложения и умножения, к которым происходит обращение при выполнении каждого из действий. Оптимальнее было бы хранить двумерный массив, или, при использовании STL, вектор, элементы которого – строки соответствующей таблицы. Линейное представление данных снижает читабельность кода и увеличивает его объем.

К недостаткам также нужно отнести непростую реализацию классов в смысле значения каждого из методов. В реализации есть методы, которые служат для дополнения, реализации других, главных, методов. Например перегруженный оператор $+$ вовсе не возвращает верный результат сложения: он не учитывает

переполнения и пригоден только для сложения неотрицательных чисел. Для реализации действия сложения над числами служит метод `summ`, принимающий два числа типа `Finit&` и возвращающий новый объект типа `Finit` являющийся суммой двух входных, с учетом переполнения и знаков входных объектов. Аналогично, оператор `<` сравнивает не совсем правильно числа из $\mathbb{Z}_8^8$, он сравнивает только их модули. Поэтому в реализации всюду используется флаг, показывающий знак числа, и меняющийся при тех или иных действиях над числом. Конечно, такие особенности реализации вводят в заблуждение программиста, ознакамливающегося с кодом реализации.

Еще одним недостатком является реализация действий умножения и деления с большими числами. Данные действия реализованы на основе сложения и вычитания соответственно. Поэтому при желании умножить два больших числа или разделить одно большое число на намного меньшее его число, эти действия будут выполняться долго. А если увеличить разрядность арифметики, то временные затраты становятся еще большими.

Данная реализация также может быть подвержена масштабируемости. Например, на основе уже определенных действий можно ввести новые действия, такие как возведение числа в степень, нахождение НОД или НОК для двух положительных чисел. Первое может быть реализовано с помощью действия умножения, нахождение НОД двух положительных чисел может быть реализовано с помощью вычитания и его применения в алгоритме Евклида, и, наконец, НОК двух чисел всегда можно восстановить, зная НОД двух чисел и их произведение.

Еще один способ масштабировать реализацию – это осуществить возможность работы калькулятора с запоминанием, то есть реализовать способность сохранения результата и дальнейшего действия в зависимости от выбора пользователя: ввести второе число с совершить над ними какие-либо действия, или заново ввести два числа.

Список литературы

- [1] Новиков Ф. Дискретная математика. 2-е издание. СПб.: Питер, 2013. 432 с.
- [2] Кук Д., Бейз Г. Компьютерная математика Пер. с англ. М.: ФИЗМАТЛИТ, 1990. 384 с.
- [3] Горбатов В.А. Фундаментальные основы дискретной математики. М.: ФИЗМАТЛИТ, 2000. 540 с.